Artorias: IoT Security Testing Framework

Bryer Jeannotte
Network and Computer Security
SUNY Polytechnic Institute
Utica, New York 13502
Email: bryer.jeannotte@sunypoly.edu

Ali Tekeoglu
Network and Computer Security
SUNY Polytechnic Institute
Utica, New York 13502
Email: ali.tekeoglu@sunypoly.edu

***Abstract*—Products that are connected and controlled remotely via the Internet are becoming more prevalent, and as a result, homes and businesses have ever increasing attack surfaces on their networks. To raise the standard of security, companies need more ways to automate testing of the most commonly found issues with IoT devices against their products. The creation of an automated security testing framework, such as Artorias, allows companies to automate the process of finding easy to identify problems, saving time for more advanced and manual processes of security and functional testing on a product. To show the framework's stability and reliability, things such as proper coding standards should be adhered to, proper management of the project should be present, and unit-tests or code coverage of the framework should be created and maintained. This ongoing project bridges together the principles of security and proper planning and development of a larger programming effort. In this paper, we introduce the IoT Security Testing Framework *Artorias*, and present an extensible framework that interested research community members can extend for more coverage on privacy and security issues of IoT devices.**

***Index Terms*—IoT devices, Security, Privacy, Framework, IoT testbed**

I. Introduction

The main purpose of this larger coding effort, Artorias, is to increase the baseline of security in IoT devices by providing a way to more easily detect the common vulnerabilities of IoT devices. But, the reaches of IoT security do not stop at device security; web, mobile and cloud interfaces, network services, local storage, 3rd party APIs and more have to be considered besides just physical interfaces and firmware on the device itself. The broad scope of IoT security stems from the wide range of needs from both industrial and residential consumers and their intended use and environments for these devices [1]. The Open Web Application Security Project, or OWASP, evaluated and derived a list of the top 10 vulnerabilities associated with IoT devices in 2014; insufficient authentication or authorization, insecure web interfaces, lack of transport encryption, insecure network services, privacy concerns, insecure cloud interfaces, insufficient security configuration, insecure software or firmware and poor physical security, insecure mobile interfaces [2]. This list gives good direction for what should be automated by Artorias to search for while testing IoT devices. After introducing Artorias and information about it, this paper will go over what areas of the top 10 Artorias tests, how it tests those areas, how to setup and run Artorias and how Artorias should be used.

II. Background

An Internet-of-Things (IoT) device could be any pyhsical object with the following properties; (i) Capability to interact and communicate over the internet. This could be directly or indirectly through a gateway. (ii) Ability to monitor, measure, observe or control a pyhsical object, which could be itself or another object in the vicinity. An IoT is a network of devices, that are connected to internet and uniquely identifiable. These devices usually interconnect virtual and pyhsical things based on the existing infrastructure and emerging protocols, communication and information technologies. Based on these proporties, anything physical can be an IoT device. With today's technology, any physical object can be connected to the internet with appropriate electronic interface.

Most common IoT devices today are smart-phones, tablets and modern cars, since these devices are connected to internet at all times. Other ubiquitous IoT devices include various consumer devices like smart-watches, IP cameras, smart light bulbs, fitness trackers, HDMI sticks, smart thermostats, remote controlled door locks, window and door sensors, smart sprinkler systems, smart garage kits, smart home security systems, connected blood pressure monitors, smart light switches, smart ovens/microwaves, intelligent baby monitors; the list goes on and grows daily.

Number of IoT devices is expected to grow exponentially in the near future, as predicted by the analyts. For instance, according to Gartner research, 25 billion by 2020 is expected to be reached for total number of connected things. Another research report by Cisco's Internet of Things Group (IOTG), estimates the total number of internet connected devices in the order of 50 billion by 2020. To evaluate the adoption rate and interest among the public in IoT devices; Google's Chromecast which transforms a traditional TV with an HDMI port into a smart TV is a prime example. During the first 2 years on consumer market, Chromecast was sold more than 20 million. In the near future, tens of billions of IoT devices is expected to be connected to the internet according to estimates.

There are multiple factors behind the success and proliferation of IoTs recently. Availability of higher internet bandwidth, decline in the cost of electronics, adoption of cloud technologies are among the factors. All these improvements sustained the decreased cost of developing an IoT device for the consumer market. For instance, Chromecast device costs

about $35, a cloud based internet connected IP camera can be purchased for $60 and variety of fitness trackers can be obtained for about $50. Internet connected smart thermometers, that can be accessed remotely, costs about $40. This trend in cost reduction shows that, in the near future connected devices will have a slight price difference compared to legacy systems, that are not considered smart.

When it comes to security and privacy features embedded into these popular IoT devices, however, the picture looks grim. Development and production of various smart devices over a short period of time, hindered security and privacy considerations. It is known that, for many manufacturers, especially for smaller start-up companies with limited dedicated security engineering resources, time to market is more vital than having a rigorously tested device with meticulously developed security and privacy features.

Security researchers from Symantec tested the security posture of 50 off-the-shelf connected home devices and found multiple privacy and security issues. Main problems are in the lack of encryption in mobile apps accompanying the devices. In addition certificate based mutual authentication between client (IoT device) and cloud server was not found, which eases the man-in-the-middle attacks. Also, ten security issues stemming from fifteen IoT device connected web applications were found by Symantec researchers.

Another security investigation by HP researchers, consisting of 10 consumer IoT devices have found that 70% of devices used network services in clear text. Among these tested devices, 80% did not dictate sufficiently complex and lengthy passwords on top of several other serious privacy and security issues.

Studies show that serious security issues of these devices indicate the pressing need for a comprehensive security and privacy evaluation framework to ease and simplify the investigation of IoT devices.

III. Related Work

In this section, recent related work, to the best of our knowledge, on security and privacy issues of IoT devices is presented.

With the growing popularity of various IoT devices in the last decade, coupled with publicity of uncovered privacy and security issues, took the attention of wider community of security researchers [3]. A literature search reveals multiple research surveys on these devices, along with research papers proposing security oriented designs for communication protocols and IoT architectures.

Authors in [4], provide an analysis of existing as well as potential vulnerabilities and security threats against the IoT heterogeneous environment and propose potential approaches to develop secure IoT architecture.

Vulnerabilities and security challenges of various layers with respect to principles of data confidentiality, integrity and availability are surveyed in [5]. Researchers concentrated in IoT inherent issues in this survey.

The security and privacy of the main IoT frameworks are surveyed in [6]. For each of the 8 frameworks researchers investigated, they clarified fundamentals of implementing third-party smart applications, chosing trusted hardware, and embedding security features.

Researchers discussed and shared initial outcomes of security and privacy evaluation of cloud based multimedia streaming device, Chromecast, in [7] and [8].

IoT devices are designed to run with limited resources, and similarly HDMI sticks operates a simplified version of Android operating system.

Even though research on Android security is well established, most of the researceres focus on full version of Android operating system. In [9], researchers investigated more than 1000 Android specific malware, which were re-packaged versions of benign applications embedded with malicious scripts. A comprehensive study proposed two programs for Android security; Taintdroid [10] and Kirin [11]. Former program inspects Android application behavior in order to find malicious use of private information. For mitigating installation of malware, an application called Kirin is proposed, which is a lightweight security service. In another research, against more than 1,000 Android apps, static analysis was executed [12]. It revealed widespread misuse of private information and detected advertising networks. For an extended study on Android Security Framework, interested readers are referred to [13].

IP cameras are one of most notorious IoT devices with respect to security postures. Foscam wireless internet connected IP camera, was found to have a serious vulnerability [14]. With the exploitation of this vulnerability, a remote attacker was able to hijack the camera and modify firmware with malicious code. Trendnet IP cameras are also popular. A researcher found that it is possible to access the camera's live-stream remotely, by exploiting this vulnerability [15]. Dropcam is another mainstream IP camera, mostly purchased by home users. A demonstration of hacking of a Dropcam was showcased in DefCon'22 conference [16]. In this exploitation, Dropcam case was taken apart to reach the PCB board. Researchers attached cables UART pins, which exposed a root shell to the firmware. Then, they were able modiy and flash a malicious firmware to Dropcam IP Camera. Even though their exploitation required physical access, usually these devices are not pyhsically out of reach of an insider attacker. Researchers investigated network traffic captured from another IP camera by Belkin, marketted as NetCam. One of the issues discovered was that NetCam only obfuscates MJPEG images by adding random bytes preceeding image payload inside network packets and the images were transferred to cloud in clear-text [17].

A research by Center for Secure Design details principles of secure design for an activity tracker. Following secure design principles in each step of the development, they proposed an imaginary device named WearFit [18] to set the example for future activity tracker manufacturers. Privacy and security issues in Fitness trackers are studied in [18], where popular activity trackers along with their mobile applications are

investigated. Analysis of self-tracking devices [19], such as smartwatches and activity trackers as well as their accompanying mobile application, has identified privacy and security risks. Vulnerablity to location tracking was the most significant finding in this Symantec study.

Research study [20] conducted by Hewlett-Packard, involved popular Smartwatches. They found that all of the investigated popular device models contained severe vulnerabilities. Lack of user authentication and authorization were most comming problems among Smartwatches and 70% had security issues with firmware update.

IV. Artorias

Artorias is designed to run in a local network to test various IoT devices for security issues. Figure 1 shows a sample test-bed where Artorias can access IoT devices. Artorias, drives security scanners to test for security of devices. At this time, it is integrated with scanners such as OWASP Zap, Nikto, Nmap. Due to it's modular architecture, it could be extended to work with more scanners. An interaction diagram of Artorias driving other security scanners is shown in Figure 3.

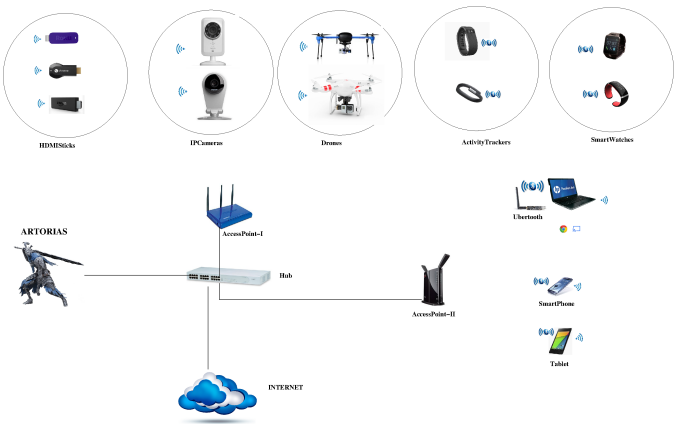


Fig. 1. Artorias in a local network test-bed with IoT devices

A. Source Code & Software Design

The code for Artorias is made public and can be found on GitHub page, https://github.com/Burrch3s/artorias. Artorias is mainly comprised of Python with some Bourne Again Shell, or Bash, for installation scripts. While developing Artorias, proper coding standards and conventions were upheld to ensure quality and correctness. For Python, PEP8, https://www.python.org/dev/peps/pep-0008/, was referred to. To ensure we were staying compliant with it, linters such as flake8 and pylint were run while coding and before merges and corrections were made before committing a change. Although Bash does not necessarily have coding standards, the linter and static code analysis tool named Shellcheck was used in the same manner as the other linters to ensure correctness of Bash code. Artorias requires the Python version 3.5 or greater for use. This was decided since the complete support for Python2 will not last forever and personal preference for the newer Python 3 over 2. To verify the integrity of the code that makes up Artorias, we also included unittests when developing. Unittests will verify the reliability of Artorias code and having a high percentage of unittest coverage will also prove its reliability. When changes were made to the Artorias code, unittests additions or changes were added as well, or soon after. Unittesting is run with the pip installable tool named Green, and unittests were written in Python with the Python Standard unittest module. Unittests can be found under the unittest directory of Artorias root source.

Figure 2 depicts high level overview of the python classes and their interaction in Artorias.

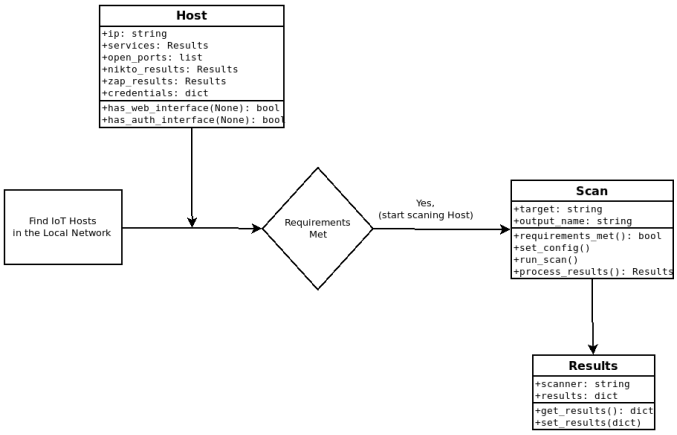


Fig. 2. Summary of python classes and their interaction in Artorias

To keep up with current and proper coding design, we tried designing with Object Orientation Paradigms in mind and represented the abstract concept of the target or host as an object, as well as the results of scanners. To draw attention to users, and make finding important messages in the output of Artorias easier, we decided to have output to the user be color coded with a short timestamp. This surely would be of use to users instead of huge output blocks of black and white in a terminal. With all these features and ideas of what should comprise Artorias, organization of the project needed to happen as well. On GitHub page for Artorias; features, bugs and enhancements are tracked as issues. They are assigned to different milestones or projects to keep track of everything and stay organized.

B. Installation and Running

Artorias is written in Python; because of this there are many functions and technologies of the project that do not have to be implemented from scratch and can be used from preexisting, verified modules in the Python Package Index, or PyPI. Artorias leverages the use of external modules to avoid "reinventing the wheel" when certain functionality is needed in the project. This guide assumes that the user is using an Ubuntu OS and that python3 as well as pip for python 3 are installed. If a different version of Linux is used, the package manager and some packages may be different, so adjust accordingly. Also, note that Python is compatible on

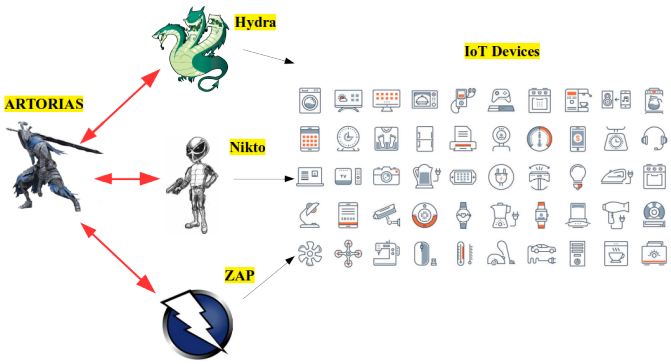


Fig. 3. Artorias interaction with various scanners

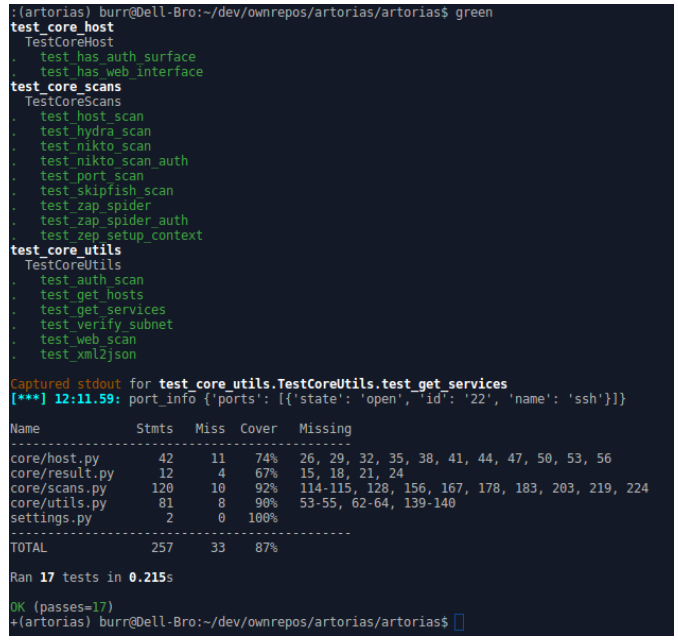
Mac and Windows operating systems, so, in theory, Artorias should also be usable on those systems as well with the correct python version, package versions, and scanners used, though it has not been tried and tested yet.

1) Installation: First, the program `git` will have to be installed. Running the command "$ apt-get install -y git" will install git for you to use. Now, we can clone the Artorias repo, "$git clone git@github.com:Burrch3s/artorias.git. Next, the user should install `virtualenv` so the Python packages required by Artorias are not installed to the system Python packages; this step is optional, but recommended to avoid any conflicting package versions. "$pip install virtualenv; virtualenv artorias_packages; source artorias_packages/bin/activate" will first install virtualenv, then create the virtual environment called artorias_packages, an arbitrary name, then we will activate or enable this environment. Now we can go into the Artorias directory and install the required packages by hand by running "$ cd artorias; pip install -r requirements.txt, or by running the `install.sh` script as root, "$ sudo ./install.sh. Now we are ready to install any scanners that are needed by the Artorias framework. Currently, the list of scanners are `Hydra`, `OWASP-Zap`, `Nikto` and `Nmap` and the wordlist `rockyou.txt`. We can install these scanners automatically by changing directories to the scanners directory and running the `install_scanners.sh` script, "$ cd scanners; sudo ./install_scanners.sh. If any of these fail to install, you should be able to research installing them by hand easily with Google. If you wish to change the output of scanner reports or the wordlist used, simply adjust the variables in the `settings.py` file located in the root of the `Artorias` project.

2) Unit-tests: After installation, the program `green` should have been installed with `pip`. To run unittests and verify the code coverage and passing of unittests, simply run $ green from the root of the Artorias repo. An example output of the latest unittest run on the most current version of Artorias is shown in Figure 4.

C. Running Artorias

The Python module `argparse` is used with Artorias to make specifying command line arguments easier and less ambiguous for the user. If a user doesnt know the arguments



```
:(artorias) burr@Dell-Bro:~/dev/ownrepos/artorias/artorias$ green
test_core_host
  TestCoreHost
.    test_has_auth_surface
.    test_has_web_interface
test_core_scans
  TestCoreScans
.    test_host_scan
.    test_hydra_scan
.    test_nikto_scan
.    test_nikto_scan_auth
.    test_port_scan
.    test_skipfish_scan
.    test_zap_spider
.    test_zap_spider_auth
.    test_zap_setup_context
test_core_utils
  TestCoreUtils
.    test_auth_scan
.    test_get_hosts
.    test_get_services
.    test_verify_subnet
.    test_web_scan
.    test_xml2json

Captured stdout for test_core_utils.TestCoreUtils.test_get_services
[***] 12:11.59: port_info {'ports': [{'state': 'open', 'id': '22', 'name': 'ssh'}]}

Name              Stmts   Miss  Cover   Missing
-----------------------------------------------
core/host.py         42     11    74%   26, 29, 32, 35, 38, 41, 44, 47, 50, 53, 56
core/result.py       12      4    67%   15, 18, 21, 24
core/scans.py       120     10    92%   114-115, 128, 156, 167, 178, 183, 203, 219, 224
core/utils.py        81      8    90%   53-55, 62-64, 139-140
settings.py           2      0   100%
-----------------------------------------------
TOTAL               257     33    87%

Ran 17 tests in 0.215s

OK (passes=17)
+(artorias) burr@Dell-Bro:~/dev/ownrepos/artorias/artorias$
```

Fig. 4. Unit Tests output

to specify, they can run "-h" to display the help information as well. Currently, the only "command" that can be run is test, which will run every available scan against a target, targets, or subnet of targets. Running "$ ./artorias test -t 192.168.0.2" will run every scan on the host located at the IP 192.168.0.2. A space separated list can also be given to this argument. A subnet can be tested instead with the -s argument and nmap will discover hosts for testing.

With the current implementation of Artorias, dynamic execution of Web Application and Authentication/Authorization scans can be executed. First identified targets or subnets are specified. In a subnet, a host scan is executed, then the given hosts are scanned with Nmap for listening services or ports on each individual target. The output of Nmap is saved to the output variable located in the settings.py file and formatted to XML. Artorias then converts this XML file essentially to JSON or a dictionary and determines if further scanning can be performed, firstly if there are appropriate ports for running Hydra to crack potential credentials and if there exists a Web Application in which to run Nikto and OWASP-Zap.

D. What does Artorias Test

Currently, using `Nikto`, `OWASP-Zap`, `Nmap`, and `Hydra`, Artorias detects vulnerabilites in the OWASP top 10 categories, I1, I2, I3 I4 and I9 [2]. This coverage is achieved by Nmap verifying the OS fingerprint, open or listening ports and sometimes versions of services, Nikto and Zap discovering Web Application issues and Hydra discovering any weak dictionary based credentials on services such as telnet, SSH, FTP or web services.

In the current version of Artorias, any credentials or vulnerabilities discovered are just logged to the user, but in the future ranking of issues as well as the displaying of the OWASP category will be logged as well.

V. How should artorias be used

Artorias should be integrated into the IoT development cycle to reduce vulnerabilities by detecting earlier on issues in the design or development of a device. Development cycles consist of variations of the design, develop, integrate, test phases, and sometimes repeated iterations of the test phase after other phases. By including the execution of the Artorias framework in the test phase, a security regression test can be achieved due to Artorias possibly finding potential issues and recording them, developers fixing the issues and then in the next batch of testing vulnerabilities can be verified as gone or having resurfaced.

For this use case, the development of the "test" command in Artorias was made first and running individual scans was left for later. we felt it more important to design and finalize the development of Artorias being able to be used in this "test everything at once" sort of manner. "Junk Hacking" or the testing and identifying of vulnerabilities in common or household objects, was not the main purpose of this paper and project, but an attempt at developing a solution to automate security testing in IoT devices. As more links from the physical to the virtual world are made, the attack surfaces of networks both residential and corporate alike will increase.

VI. Challenges to the project

There were some challenges with setting out to accomplish and finish Artorias. First, the overall difficulty in designing and maintaining of a larger code base can be a challenge. Taking the time to stop and consider possible ways to do or implement something can prove difficult. we were able to receive some experience in these areas simply by trying and sometimes failing in this area. Next, the current amount of scanners that target specifically IoT devices are not very high. For the purpose of IoT devices, and the huge attack surface from device security, to network surface, to the myriad of different possible interfaces, to make a "one for all IoT scanner is simply not feasible, and perhaps why there are not many IoT scanners currently available. To find and utilize scanners for IoT we had to think more outside the box and utilize common Web App scanners, for example, an leverage them instead to aid testing of IoT devices. This was one of the first identifiers to me that Artorias should be a framework instead and leverage scanners that werent unique to IoT device security.

VII. Conclusion

Studies show that serious security and privacy issues commonly found in Internet of Things Devices indicate the pressing need for a comprehensive security and privacy evaluation framework to ease and simplify the investigation of IoT devices. Artorias is designed to run in a local area network to test various IoT devices for potential security and privacy issues. Due to its modular architecture, it could be extended to work with more scanners. It is designed to check for most common problems in IoT devices, and other software accompanying the devices, such as mobile application, cloud communication and web interface.

References

[1] D. Caivano, F. Cassano, R. Lanzilotti, and A. Piccinno, "Towards an IoT Model for the Assessment of Smart Devices," in *Proceedings of the 2018 International Conference on Advanced Visual Interfaces, AVI 2018, Castiglione della Pescaia, Italy, May 29 - June 01, 2018*, 2018, pp. 57:1–57:3. [Online]. Available: https://doi.org/10.1145/3206505.3206587

[2] Miessler, D., Smith C., "Top 10 IoT Vulnerabilities (2014," "https://www.owasp.org/index.php/OWASP_Internet_of_Things_Project", [Online]; accessed 31-Dec-2018.

[3] A. Tekeoglu and A. S. Tosun, "An Experimental Framework for Investigating Security and Privacy of IoT Devices," in *Intelligent, Secure, and Dependable Systems in Distributed and Cloud Environments - First International Conference, ISDDC 2017, Vancouver, BC, Canada, October 26-28, 2017, Proceedings*, 2017, pp. 63–83. [Online]. Available: https://doi.org/10.1007/978-3-319-69155-8_5

[4] F. A. Alaba, M. Othman, I. A. T. Hashem, and F. Alotaibi, "Internet of Things Security: A Survey," *Journal of Network and Computer Applications*, vol. 88, pp. 10–28, 2017. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S1084804517301455

[5] D. M. Mendez, I. Papapanagiotou, and B. Yang, "Internet of Things: Survey on Security and Privacy," jul 2017. [Online]. Available: http://arxiv.org/abs/1707.01879 http://dx.doi.org/10.1080/19393555.2018.1458258

[6] M. Ammar, G. Russello, and B. Crispo, "Internet of Things: A survey on the security of IoT frameworks," *Journal of Information Security and Applications*, vol. 38, pp. 8–27, feb 2018. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2214212617302934

[7] A. Tekeoglu and A. Tosun, "Blackbox Security Evaluation of Chromecast Network Communications," in *Performance Computing and Communications Conference (IPCCC), 2014 IEEE Int.*, Dec 2014, pp. 1–2.

[8] ——, "A Closer Look into Privacy and Security of Chromecast Multimedia Cloud Communications," in *Int. Workshop on Multimedia Cloud Comm., 2015 IEEE INFOCOM*, April 2015, pp. 121–126.

[9] Y. Zhou and X. Jiang, "Dissecting Android Malware: Characterization and Evolution," in *IEEE Symposium on Security and Privacy*, 2012.

[10] W. Enck, P. Gilbert, B.-G. Chun, L. P. Cox, J. Jung, P. McDaniel, and A. N. Sheth, "TaintDroid: An Information-flow Tracking System for Realtime Privacy Monitoring on Smartphones," in *Proceedings of the 9th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'10. Berkeley, CA, USA: USENIX Association, 2010.

[11] W. Enck, M. Ongtang, and P. McDaniel, "On Lightweight Mobile Phone Application Certification," in *Proceedings of the 16th ACM Conference on Computer and Communications Security*, 2009, pp. 235–245.

[12] W. Enck, D. Octeau, P. McDaniel, and S. Chaudhuri, "A Study of Android Application Security," in *Proceedings of the 20th USENIX Conference on Security*, ser. SEC'11. Berkeley, CA, USA: USENIX Association, 2011.

[13] W. Enck, M. Ongtang, and P. McDaniel, "Understanding Android Security," *IEEE Security and Privacy*, vol. 7, no. 1, pp. 50–57, Jan. 2009. [Online]. Available: http://dx.doi.org/10.1109/MSP.2009.26

[14] S. Shekyan and A. Harutyunyan, "To Watch or Be Watched: Turning Your Surveillance Camera Against You," "http://conference.hitb.org/hitbsecconf2013ams/shekyan-harutyunyan/", 2016.

[15] "Trendnet Cameras - I always feel like somebody's watching me," "http://console-cowboys.blogspot.com/2012/01/trendnet-cameras-i-always-feel-like.html", 2016.

[16] P. Wardle and C. Moore, "Optical Surgery: Implanting a Dropcam," Synack Labs, Defcon 22 Hacking Conference, Aug 2014, "https://www.synack.com/labs/projects/implanting-a-dropcam".

[17] A. Tekeoglu and A. Tosun, "Investigating Security and Privacy of a Cloud-Based Wireless IP Camera: NetCam," in *5th International Workshop on Privacy, Security and Trust in Mobile and Wireless Systems (MobiPST), 2015 IEEE ICCCN*, August 2015, pp. 1–6.

[18] J. West, T. Kohno, D. Lindsay, and J. Sechman, "Wearfit: Security design analysis of a wearable fitness tacker," IEEE Center for Secure Design, Tech. Rep., 2016.

[19] M. Barcena, C. Wueest, and H. Lau, "How safe is your quantified self," Symantec Security Response, Tech. Rep., 2014.

[20] "Internet of things security study: Smartwatches," Hewlett-Packard, Tech. Rep., 2015.